

Identity MVC

Programming Integration

1	HET MICROSOFT IDENTITY PLATFORM	4
1.1	Inleiding	4
1.2	Identity configureren in een bestaand MVC Project	4
1.3	Nuget packages	5
1.4	Configuratie in van de application Dbcontext	5
1.5	Identity registreren en configureren in program.cs	6
1.6	Migration genereren en database updaten	7
1.7	IdentityUser uitbreiden	8
1.7.1	Aanpassen Program klasse	8
1.7.2	Aanpassen ApplicationDbContext	9
2	REGISTEREN EN INLOGGEN IN EEN MVC APPLICATIE	10
2.1	Accountregisterviewmodel	10
2.2	Accountcontroller	11
2.3	View aanmaken	11
2.4	Register logic uitwerken	12
2.5	Accountloginviewmodel	14
2.6	Login View en action methods	14
3	AUTHORISATIE IN EEN MVC APPLICATIE	17
3.1	MembersZoneController:	18
3.2	MembersZone Index View:	18
3.3	MembersZone MemberBenefits View:	18
4	ROLE GEBASEERDE AUTHORISATIE	19
4.1	IdentityRole	19
4.1.1	IdentityRole properties	19
4.1.2	IdentityRole methods	19
4.1.3	IdentityRole tabel in database	20
4.2	IdentityUserRole<TKey>	20
4.2.1	IdentityUserRole Properties	20
4.2.2	IdentityUserRole tabel in database	20
4.3	Admin role aanmaken	21
4.3.1	Admin user en admin role seeden	22
4.3.2	Migration en database update	23

4.3.2.1	AspNetUsers	23
4.3.2.2	AspNetRoles	23
4.3.2.3	AspNetUserRoles	23
4.4	UsersController enkel voor admin	23
5	APPENDIX	27
5.1	EmailConfirmation	27
5.1.1	Een token genereren	27
5.1.2	De validateemail methode	27
5.1.3	Email versturen met mimekit	29
5.1.3.1	Secrets.json	29
5.1.3.2	De confirmationlink	30
5.1.3.3	E-mail versturen	31
5.2	Identity CRUD voor users en roles	33
5.2.1	IdentityUser	33
5.2.2	userManager<TUser>	33
5.2.3	RoleManager<TRole>	34
5.2.4	IdentityResult	34
5.2.5	SignInManager<TUser>	35
5.2.6	Eenvoudig rollenbeheer	35
5.2.6.1	UsersController	35
5.2.6.2	De Index methode	36
5.2.7	Nieuwe rol toevoegen	38
5.2.8	Een role toevoegen aan een gebruiker	40
5.2.9	Een role verwijderen van een gebruiker	41

1 HET MICROSOFT IDENTITY PLATFORM

1.1 INLEIDING

Microsoft Identity Platform is een lidmaatschapssysteem dat login functionaliteit kan toevoegen aan .Net apps. Gebruikers kunnen een account maken met de in Identity opgeslagen inloggegevens of een externe login provider gebruiken. Standaard ondersteunt Identity volgende externe aanmeldingsproviders: Facebook, Google, Microsoft Account en Twitter.

Je kan Identity configureren om met behulp van een SQL Server-database gebruikersnamen, wachtwoorden en profielgegevens op te slaan. Als alternatief kan je uiteraard opteren om met andere mogelijkheden aan de slag te gaan, bijvoorbeeld [Azure Table Storage](#).

In dit deel van de cursus volgen we de standaard leidraad van Microsoft over hoe je Identity gebruikt, configureert en verder aanpast naar je eigen wensen. Deze documentatie is tevens ook beschikbaar via de [Microsoft docs](#). De vernieuwde *learning site* van Microsoft.

Dankzij de open source kaart die Microsoft getrokken heeft is de volledige broncode van asp.net Identity ook terug te vinden op [GitHub](#).

1.2 IDENTITY CONFIGUREREN IN EEN BESTAAND MVC PROJECT

Wanneer we een authenticatiesysteem willen implementeren in een bestaande MVC applicatie dienen we onderstaande stappen te doorlopen. We gebruiken als voorbeeld een eenvoudige Mvc applicatie die informatie voorstelt van een schoolsysteem.



Code repository

De startsituatie van dit hoofdstuk is te vinden op

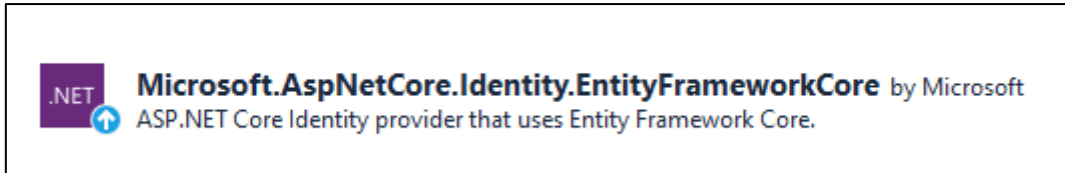
 `git clone` <https://classroom.github.com/a/-KFuUxce>

Nadat je het project gecloned hebt kan je de initiële database opzetten met onderstaand commando in de package manager console:

```
dotnet ef database update First -s .\RateACourse.Web -p .\RateACourse.Core
```

1.3 NUGET PACKAGES

De eerste stap is het installeren van de juiste nuget packages. We voegen onderstaande package toe aan het web project:



In het huidige project kiezen we voor *versie 8.0.xx*.

1.4 CONFIGURATIE IN VAN DE APPLICATION DBCONTEXT

Microsoft Identity gebruikt een database om de nodige gegevens over gebruikers bij te houden. Om dit toe te passen op onze bestaande applicatie gaan we de Db context moeten aanpassen en onderstaande code moeten wijzigen in de *CourseRateDbContext* class:

```
public class CourseRateDbContext : DbContext
```

wordt:

```
public class CourseRateDbContext : IdentityDbContext
```

Het overerven van IdentityDbContext zorgt ervoor dat allen nodige tabellen toegevoegd zullen worden bij het aanmaken van een volgende migration.

1.5 IDENTITY REGISTREREN EN CONFIGUREREN IN PROGRAM.CS

Om correct te kunnen authenticeren met Microsoft Identity dienen we de pipeline in *Program.cs* eveneens te configureren.

We voegen onderstaande code toe, net voor de `builder.Services.AddControllersWithViews()` aanroep.



Opgelet

Deze configuratie is helemaal niet geschikt om in productiecode te gebruiken. We doen dit enkel om vlot te kunnen testen.

```
//add identity
builder.Services.AddIdentity<IdentityUser, IdentityRole>(options =>
{
    //configure options for testing purposes
    options.Password.RequiredUniqueChars = 0;
    options.Password.RequireNonAlphanumeric = false;
    options.Password.RequireDigit = false;
    options.Password.RequireLowercase = false;
    options.Password.RequireUppercase = false;
    options.Password.RequiredLength = 6;
}).AddEntityFrameworkStores<CourseRateDbContext>()
.AddDefaultTokenProviders();
```

De `builder.Services.AddIdentity` methode configureert de services die verantwoordelijk zijn voor gebruikersauthenticatie en -autorisatie in je applicatie. Het registreert die services bij de dependency container van je applicatie, waardoor je toegang krijgt tot verschillende functionaliteiten van ASP.NET Core Identity. De belangrijkste services die geregistreerd worden zijn:

- De *UserManager* service: om gebruikersbeheer uit te voeren.
- De *SignInManager* service: om in en uitloggen mogelijk te maken.
- De *RoleManager* service: om het oude rollensysteem te beheren (is minder van toepassing in deze module)
- De nodige *CookieSchemes* om het authenticatiesysteem te doen werken. (Application, External en 2FA worden standaard geconfigureerd)

De `AddEntityFrameworkStores<CourseRateDbContext>` zorgt ervoor dat Identity weet welke database er moet gebruikt worden om de tabellen te genereren en de informatie op te slaan en op te halen.

Tot slot zal de `AddDefaultTokenProviders()` methode standaard tokenproviders toevoegen aan ASP.NET Core Identity. Deze tokenproviders stellen je in staat om verschillende soorten tokens te genereren en te gebruiken voor verschillende doeleinden, zoals wachtwoordreset, e-mailverificatie, tweefactorauthenticatie, ...

1.6 MIGRATION GENEREREN EN DATABASE UPDATEN

Eens we voorgaande zaken geconfigureerd hebben kunnen we het commando ingeven om een migration te genereren:

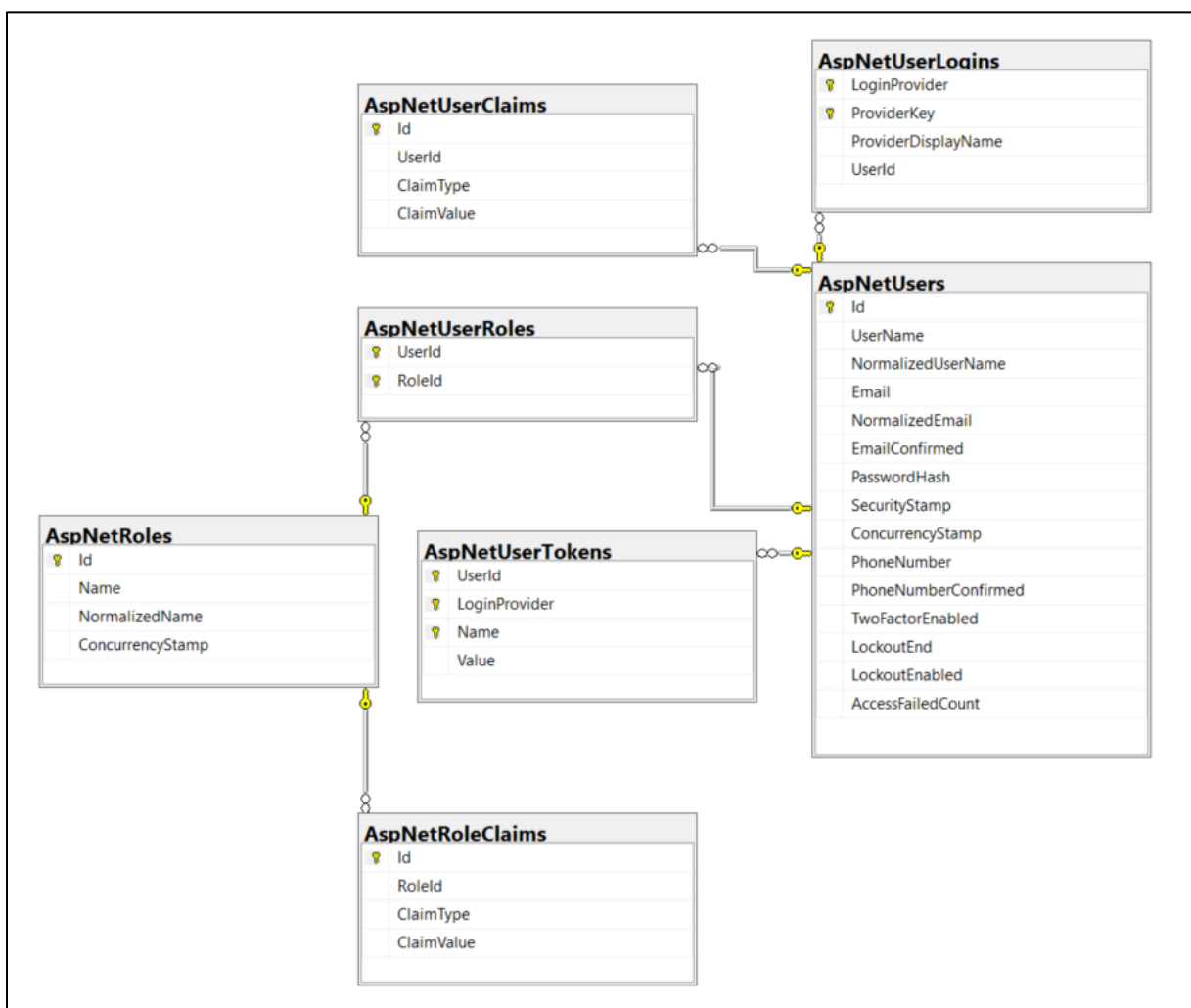
```
dotnet ef migrations add Identity -s .\RateACourse.Web -p .\RateACourse.Core
```

De -s flag bevat een verwijzing naar het project waar de connectionstring zich bevindt(Appsettings.json) en de -p flag moet het project bevatten waar de ApplicationDbContext gevonden kan worden(Core project).

Als het aanmaken zonder fouten verlopen is kan je de database updaten met onderstaand commando:

```
dotnet ef database update Identity -s .\RateACourse.Web -p .\RateACourse.Core
```

Dit zou, indien foutloos verlopen, de volgende tabellen hebben aangemaakt in de database:



1.7 IDENTITYUSER UITBREIDEN

Nu we het raamwerk opgezet hebben om authenticatie te ondersteunen door middel van het Microsoft Identity Framework kunnen we onze implementatie wat verder uitbreiden en customizen om te voldoen aan onze specifieke voorbeeldapplicatie. De standaard `IdentityUser` die voorzien wordt door Identity bevat geen verdere properties waarmee we een gebruiker kunnen identificeren. Willen we dus zaken als Voornaam, naam, adresgegevens, ... gaan bijhouden over onze gebruikers dan zullen we de standaard `IdentityUser` moeten uitbreiden met onze extra properties.

We passen hiervoor de reeds aanwezige `Student` klasse aan om deze te laten overerven van `IdentityUser`.

Gezien we van onze gebruikers ook de naamgegevens en de geboortedatum wensen bij te houden, voegen we de nodige properties toe aan de `Student` class.

```
public class Student : IdentityUser
{
    public string Firstname { get; set; }
    public string Lastname { get; set; }
    public DateTime DateOfBirth { get; set; }
    ...
}
```

Hou rekening dat je wel enkele aanpassingen zal moeten uitvoeren hierdoor.

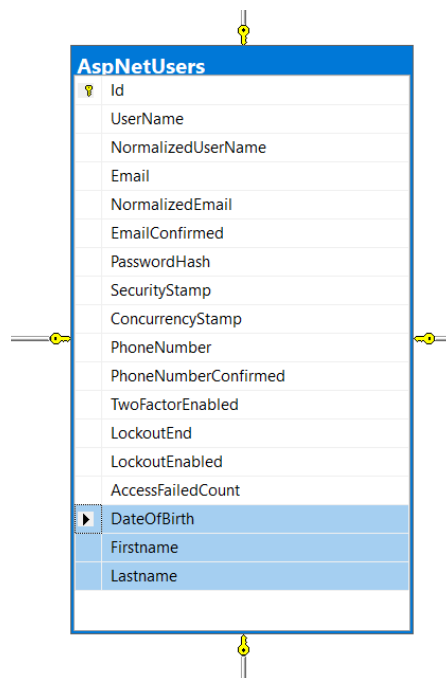
1.7.1 AANPASSEN PROGRAM KLASSE

```
//add identity
builder.Services.AddIdentity<Student, IdentityRole>(options =>
{
    //configure options for testing purposes
    ...
}).AddEntityFrameworkStores<CourseRateDbContext>()
.AddDefaultTokenProviders();
```


1.7.2 AANPASSEN APPLICATIONDBCONTEXT

```
public class ApplicationDbContext : IdentityDbContext<ApplicationUser>
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }
}
```

Vergeet hier niet een nieuwe migration toe te voegen en een update van je database uit te voeren! Hierdoor komen de extra properties van je ApplicationUser ook als kolommen in je database terecht.

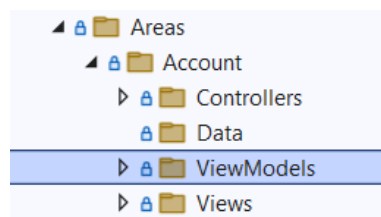


2 REGISTEREN EN INLOGGEN IN EEN MVC APPLICATIE

Nu we onze gebruiker uitgebreid hebben met de nodige properties kunnen we beginnen aan het implementeren van de register functionaliteit. We maken hiervoor eerst een aparte *Area* aan waar we een **AccountController** in zullen onderbrengen. Je kan hiervoor rechtsklikken op het web project en vervolgens linksklikken op *Add => New Scaffolded Item => Mvc Area*. Vervolgens geef je *Account* in als naam voor de nieuwe Area.

2.1 ACCOUNTREGISTERVIEWMODEL

Om een register form te maken hebben we uiteraard een viewmodel nodig. Maak een map **ViewModels** aan in de pas aangemaakte Account area.



In deze folder gaan we de viewmodels plaatsen die we in de *AccountController* gaan gebruiken. Maak een *AccountRegisterViewModel* class aan met onderstaande properties:

```
public class AccountRegisterViewModel
{
    [Required]
    [EmailAddress]
    public string Username { get; set; }
    [Required]
    public string Firstname { get; set; }
    [Required]
    public string Lastname { get; set; }
    [Required]
    [DataType(DataType.Password)]
    public string Password { get; set; }
    [Compare("Password")]
    [DataType(DataType.Password)]
    public string RepeatPassword { get; set; }
}
```

2.2 ACCOUNTCONTROLLER

Vervolgens maak je een `AccountController` aan in de controller map van de Account Area met onderstaande action methodes:

```
[HttpGet]
public IActionResult Register()
{
    return View();
}

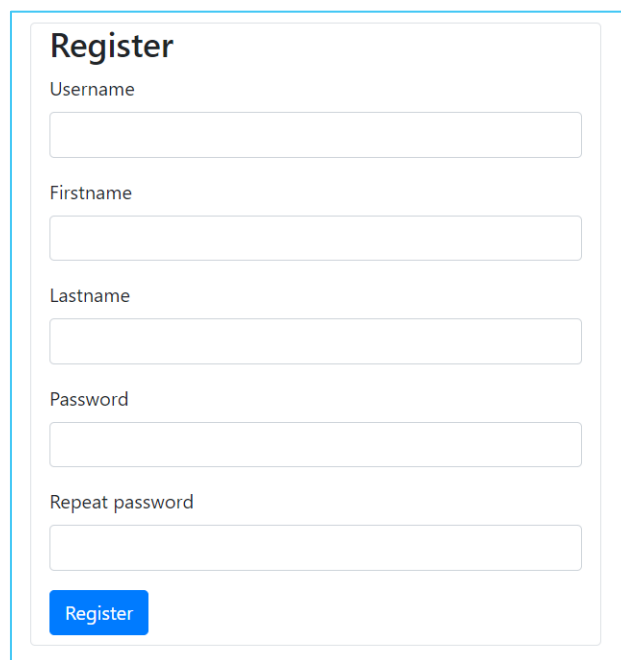
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Register(AccountRegisterViewModel
accountRegisterViewModel)
{
    return View();
}
```

2.3 VIEW AANMAKEN

Tot slot maken we de register view aan die onderstaande code bevat:

```
@model AccountRegisterViewModel
<div class="row">
    <div class="col col-lg-5 border border-1 rounded rounded-3 mx-auto">
        <h3>@ViewBag.PageTitle</h3>
        <form asp-area="Account" asp-controller="Account" asp-action="Register"
method="post">
            <div class="form-group">
                <label asp-for="Username"></label>
                <input asp-for="Username" class="form-control">
                <span class="text-danger" asp-validation-for="Username"></span>
            </div>
            <div class="form-group">
                <label asp-for="Firstname"></label>
                <input asp-for="Firstname" class="form-control">
                <span class="text-danger" asp-validation-for="Firstname"></span>
            </div>
            <div class="form-group">
                <label asp-for="Lastname"></label>
                <input asp-for="Lastname" class="form-control">
                <span class="text-danger" asp-validation-for="Lastname"></span>
            </div>
            <div class="form-group">
                <label asp-for="Password"></label>
                <input asp-for="Password" class="form-control">
                <span class="text-danger" asp-validation-for="Password"></span>
            </div>
            <div class="form-group">
                <label asp-for="RepeatPassword"></label>
                <input asp-for="RepeatPassword" class="form-control">
                <span class="text-danger" asp-validation-for="RepeatPassword"></span>
            </div>
            <button type="submit" class="btn btn-primary mb-2">Register</button>
        </form>
    </div>
</div>
```

Het Register formulier zal er dan als volgt uitzien:



The image shows a registration form titled "Register". It contains five input fields: "Username", "Firstname", "Lastname", "Password", and "Repeat password". Below the "Repeat password" field is a blue button labeled "Register". The form is enclosed in a light blue border.

2.4 REGISTER LOGIC UITWERKEN

Nu is het tijd om de register logic uit te werken. Kort samengevat zal alle code die op de één of andere manier een gebruiker authenticceert onderstaande acties uitvoeren:

- Opvragen van de gegevens die nodig zijn om te authenticeren(register form).
- Controleren of de gegevens al dan niet gekend zijn in het systeem.
- Indien de gegevens correct zijn, de gebruiker registreren binnen het systeem(database).

Puntje één wordt afgehandeld door het Register form, puntjes twee en drie zullen we moeten implementeren met behulp van Microsoft Identity, en meer bepaald door gebruik te maken de `UserManager<T>` Serviceklasse. Deze klasse wordt geregistreerd voor dependency injection door het aanroepen van de `AddIdentity<T,T>` methode in de Program.cs klasse.

We injecteren dus eerst de `UserManager<T>` klasse in de AccountController:

```
private readonly UserManager<ApplicationUser> _userManager;  
  
public AccountController(UserManager<ApplicationUser> userManager)  
{  
    _userManager = userManager;  
}
```

Vervolgens werken we de aanwezige Register methodes uit:

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Register(AccountRegisterViewModel
accountRegisterViewModel)
{
    if(!ModelState.IsValid)
    {
        return View(accountRegisterViewModel);
    }
    if (await _userManager.FindByNameAsync(accountRegisterViewModel.Username) == null)
    {
        var user = new ApplicationUser
        {
            UserName = accountRegisterViewModel.Username,
            Firstname = accountRegisterViewModel.Firstname,
            Lastname = accountRegisterViewModel.Lastname,
            Email = accountRegisterViewModel.Username,
            EmailConfirmed = true
        };
        await _userManager.CreateAsync(user, accountRegisterViewModel.Password);
        return View("Registered");
    }
    return RedirectToAction(nameof(Index), "Courses");
}
```

We gebruiken hiervoor twee methodes van de UserManager klasse. In eerste instantie controleren we of de username van de nieuwe gebruiker niet reeds gekend is in het systeem (de FindByNameAsync() methode). Vervolgens, indien de FindByNameAsync() methode null retourneerde, gaan we met behulp van de CreateAsync() methode de net aangemaakte gebruiker opslaan in de database. Merk op dat het wachtwoord als tweede parameter meegegeven wordt, dit zal ervoor zorgen dat de gebruiker met het ingegeven wachtwoord, uiteraard geëncrypteerd, opgeslagen wordt.

Let ook eventjes op de code om een nieuwe gebruiker aan te maken. De laatste regel `EmailConfirmed = true` zal ervoor zorgen dat we straks kunnen inloggen, zonder dat de gebruiker zijn emailadres bevestigde. Dit doen we nu uiteraard enkel om te testen. In het appendix hoofdstuk vind je terug hoe je de emailbevestiging op een correcte wijze kan uitwerken.

Test nu je code uit, start de applicatie en surf naar de Register methode van de AccountController. Vul het registerformulier in en, als alles zonder fouten verloopt, bekijk je het resultaat in de `dbo.AspNetUsers` tabel in de database:

UserName	NormalizedUserName	Email	NormalizedEmail	EmailConfirmed	PasswordHash
mileto1975@gmail.com	MILETO1975@GMAIL.COM	mileto1975...	MILETO1975@GMAIL...	True	AQAAAAEAAACcQAAAAEF3cWNPQ7PxbT72bbm8PwOCFvzluIOF4uPncuWeOEz/G+llme+Fwhyi27WDxj/Q...

Merk op dat het paswoord netjes gehasht is, dit werd eveneens door de `CreateAsync()` methode geregeld.

2.5 ACCOUNTLOGINVIEWMODEL

Nu het registreren van de gebruiker achter de rug is kunnen we werk maken van de loginfunctionaliteit. De eerste stap hierin is het aanmaken van een `AccountLoginViewModel`

Maak een nieuwe klasse `AccountLoginViewModel` aan in de folder *ViewModels* van de *Account* Area. Plaats daarin onderstaande code:

```
[Required]
[EmailAddress]
public string Username { get; set; }
[Required]
[DataType(DataType.Password)]
public string Password { get; set; }
```

Merk op dat deze properties overeenkomen met een aantal properties in onze `AccountRegisterViewModel`, wat ons een mooie kans biedt om wat overerving te implementeren. Verander de klasse `AccountRegisterViewModel` als volgt:

```
public class AccountRegisterViewModel : AccountLoginViewModel
{
    [Required]
    public string Firstname { get; set; }
    [Required]
    public string Lastname { get; set; }

    [Compare("Password")]
    [DataType(DataType.Password)]
    [Display(Name = "Repeat password")]
    public string RepeatPassword { get; set; }
}
```

We erven de nuttige properties over van de `AccountLoginViewModel` klasse en voegen de nodige properties erbij.

LOGIN VIEW EN ACTION METHODS

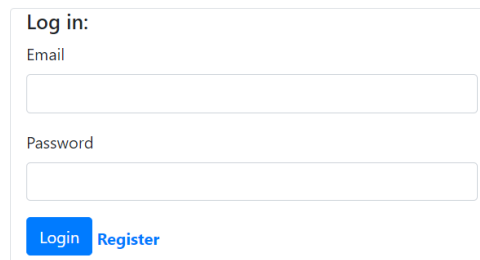
Nu kunnen we de nodige action methodes implementeren om de loginfunctionaliteit verder uit te werken. Maak in de *AccountController* de onderstaande methodes aan:

```
[HttpGet]
public IActionResult Login()
{
    ViewBag.PageTitle = "Log in:";
    return View();
}
```

Deze methode zal het loginformulier displayen. Vervolgens maak je de bijhorende view aan:

```
@model AccountLoginViewModel
<div class="row">
  <div class="col col-lg-5 border border-1 rounded rounded-3 mx-auto">
    <h5>@ViewBag.PageTitle</h5>
    <div class="text-danger" asp-validation-summary="ModelOnly"></div>
    <form asp-area="Account" asp-controller="Account" asp-action="Login"
method="post">
      <div class="form-group">
        <label asp-for="Email"></label>
        <input asp-for="Email" class="form-control">
        <span class="text-danger" asp-validation-for="Email"></span>
      </div>
      <div class="form-group">
        <label asp-for="Password"></label>
        <input asp-for="Password" class="form-control">
        <span class="text-danger" asp-validation-for="Password"></span>
      </div>
      <button type="submit" class="btn btn-primary mb-2">Login</button>
      <a asp-action="Register" class="alert-link">Register</a>
    </form>
  </div>
</div>
```

Deze view zal er in runtime zo uitzien:



Nu schrijven we de login logica uit. Plaats onderstaande code in de AccountController:

```
public async Task<IActionResult> Login(AccountLoginViewModel accountLoginViewModel)
{
    if (!ModelState.IsValid)
    {
        return View(accountLoginViewModel);
    }
    var result = await _signInManager.PasswordSignInAsync(accountLoginViewModel.Email,
accountLoginViewModel.Password, false, true);
    if (!result.Succeeded)
    {
        ModelState.AddModelError("", "Wrong credentials!");
        return View(accountLoginViewModel);
    }
    return RedirectToAction("Index", new { Area = "Admin", Controller = "Courses" });
}
```

De code in geel gemarkeerd zorgt voor de authenticatie van de gebruiker. De overloaded `PasswordSignInAsync()` methode neemt als eerste twee parameters de username en het paswoord die geauthenticeerd moeten worden. Parameter drie is een bool die aangeeft of de cookie moet persisteren, met andere woorden, of de gebruiker ingelogd moet blijven na het sluiten van de browser. Parameter vier is een bool om aan te geven dat de gebruiker een lockout moet krijgen bij een aantal verkeerde inlogpogingen.

De `PasswordSignInAsync()` methode retourneert een object van de `SignInResult` klasse. Daarmee kan je nagaan of de actie gelukt is, of de gebruiker dus correct geauthenticeerd is. Dit doen we door middel van de bool `Succeeded`. Indien deze op true staat kunnen we een `RedirectToAction` retourneren om de ingelogde gebruiker door te verwijzen naar de afgeschermd pagina, in ons geval de `AccountController.Index` methode in de Admin area.

3 AUTHORISATIE IN EEN MVC APPLICATIE

Identity laat ons niet alleen toe om gebruikers te registreren, in te loggen en uit te loggen (*Authenticatie*). We kunnen nu ook de nodige controles uitvoeren of een gebruiker wel degelijk recht heeft om een bepaalde pagina te bezoeken. We spreken in dit geval van *Authorisatie*.

De meest eenvoudige vorm van authorisatie is het bepalen van de toegang tot een pagina, resource, op basis van de *authenticatiestatus*. Met andere woorden, indien een gebruiker ingelogd is mag hij een bepaalde pagina of resource bekijken/gebruiken. Dit wordt in .Net Core toegepast door middel van het **Authorize** attribuut.

Om dit te illustreren gaan we even naar de HomeController.

Op de HomeController plaatsen we een **Authorize** attribuut net boven de Privacy action.

```
[Authorize]
public IActionResult Privacy()
{
    return View();
}
```

Vervolgens starten we onze applicatie op en trachten naar de Privacypagina te navigeren terwijl we niet ingelogd zijn.

We zien meteen dat deze ons onmiddellijk richting de Login pagina stuurt. Prima opgevangen dus!

Het **Authorize** attribuut kan echter ook op Controller niveau geplaatst worden. Als je dus een controller hebt waarvan enkel ingelogde gebruikers toegang mogen hebben kun je **[Authorize]** attribuut toevoegen boven de Controller naam in de desbetreffende controller.

Wens je echter in een Controller met het Authorize attribuut toch één of meerdere actions toegankelijk te maken voor iedereen dan kan je hiervoor het **[AllowAnonymous]** attribuut gaan toevoegen.

3.1 MEMBERSZONECONTROLLER:

```
namespace Pri.Identity.Web.Controllers
{
    [Authorize]
    public class MembersZoneController : Controller
    {
        [AllowAnonymous]
        public IActionResult Index()
        {
            return View();
        }

        public IActionResult MemberBenefits()
        {
            return View();
        }
    }
}
```

3.2 MEMBERSZONE INDEX VIEW:

```
@{
    ViewData["Title"] = "Index";
}

<h1>Index</h1>

<p>This can be viewed by anyone</p>

<a class="btn btn-danger" asp-controller="MembersZone" asp-
action="MemberBenefits">Discover our member benefits!</a>
```

3.3 MEMBERSZONE MEMBERBENEFITS VIEW:

```
@{
    ViewData["Title"] = "MemberBenefits";
}

<h1>MemberBenefits</h1>

<p>This can only be viewed by logged in users</p>
```

4 ROLE GEBASEERDE AUTHORIZATIE

Een user van een applicatie kan behoren tot één of meerdere rollen. Neem even volgende voorbeelden:

- David kan een student zijn die dan een **rol** toegewezen krijgt van: **Student**
- Siegfried kan een lector zijn die dan een **rol** toegewezen krijgt van: **Lector**
- William kan een lector zijn die **twee rollen** toegewezen krijgt: **Lector** en **Student**
- Joren kan een administrator zijn die een **rol** toegewezen krijgt van: **Admin**

In dit hoofdstuk zullen we bespreken hoe we rollen kunnen aanmaken en gebruiken. We bouwen hiervoor verder op de voorbeeldapplicatie.

4.1 IDENTITYROLE

Net zoals we de `IdentityUser` klasse hebben voor een basic Identity user hebben we ook een klasse voor roles in Identity, namelijk de `IdentityRole` klasse.

4.1.1 IDENTITYROLE PROPERTIES

De `IdentityRole` klasse bevat volgende properties:

Id	De primaire sleutel voor deze rol (Get of Set)
Name	De naam voor deze rol (Get of Set)
NormalizedName	De genormaliseerde naam voor deze rol (Get of Set)
ConcurrencyStamp	Een willekeurige waarde die moet veranderen wanneer een rol in de database wordt bewaard

Standaard wordt de `Id` property voorzien als `string` datatype. Wensen we achter een ander datatype, bijvoorbeeld een `Guid`, dan kunnen we gebruik maken van de generic variant van deze klasse.

Bijvoorbeeld:

`IdentityRole<Guid>`, hierdoor is de primary key (de `Id` dus) geen niet van het type `string` maar van het type `Guid`. Wij zullen in dit cursusvoorbeeld gebruik maken van `string` en dus van de default `IdentityRole` implementatie.

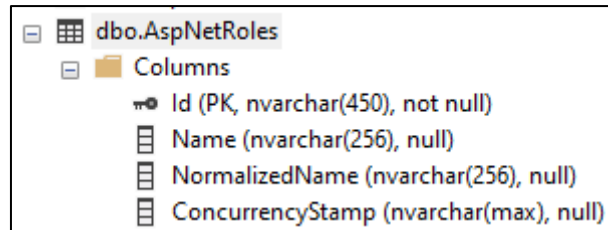
4.1.2 IDENTITYROLE METHODS

De `IdentityRole` klasse bevat volgende methode:

ToString()	Return van de naam van de rol
-------------------	-------------------------------

4.1.3 IDENTITYROLE TABEL IN DATABASE

In de database wordt een `IdentityRole` opgeslaan in de tabel `AspNetRoles`:



dbo.AspNetRoles
Columns
Id (PK, nvarchar(450), not null)
Name (nvarchar(256), null)
NormalizedName (nvarchar(256), null)
ConcurrencyStamp (nvarchar(max), null)

4.2 IDENTITYUSERROLE<TKEY>

Aangezien een `IdentityUser` meerdere `IdentityRoles` kan bevatten en een `IdentityRole` meerdere `IdentityUsers` kan bevatten zitten we dus met een veel op veel relatie.

Identity voorziet voor ons hiervoor de `IdentityUserRole` klasse.

Dit is echter een generic class die een type argument verwacht, namelijk het type van de primary keys. Aangezien wij gekozen hebben voor de default implementatie van de `IdentityUser` en `IdentityRole` zijn onze primary keys van het type string.

Hierdoor dient onze `IdentityUserRole` klasse deze ook als type argument te ontvangen:

```
IdentityUserRole<string>
```

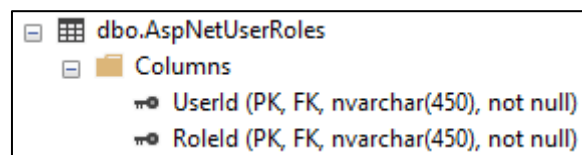
4.2.1 IDENTITYUSERROLE PROPERTIES

Deze klasse heeft volgende properties:

RoleId	De primaire sleutel van de rol die gelinkt is aan de user (Get of Set)
UserId	De primaire sleutel van de user die gelinkt is aan de rol (Get of Set)

4.2.2 IDENTITYUSERROLE TABEL IN DATABASE

In de database wordt een `IdentityUserRole<TKey>` record toegevoegd in de tabel `AspNetUserRoles`:



dbo.AspNetUserRoles
Columns
UserId (PK, FK, nvarchar(450), not null)
RoleId (PK, FK, nvarchar(450), not null)

4.3 ADMIN ROLE AANMAKEN

In dit voorbeeld zullen we een Admin role creeëren zodat we een gebruiker kunnen toevoegen aan deze Admin role.

Het is echter moeilijk om een Admin role toe te voegen aan een bestaande user als er nog geen user aanwezig is die een Admin role heeft gekregen. We willen deze functionaliteit namelijk niet beschikbaar stellen voor elke user. Maar enkel voor de users die de Admin role hebben gekregen.

Dit is dus een beetje een kip en ei verhaal; er moet eerst een user zijn met de admin role alvorens een user in onze database met admin role usermanagement kan gaan uitvoeren zoals CRUD van roles en users.

We zullen dus een Admin role toevoegen in onze database door middel van seeding en deze Admin role gaan koppelen aan een bestaande user in onze database.

Hierdoor kan dan een bestaande user met de Admin role via een applicatie roles gaan aanmaken, en gaan toekennen aan bestaande gebruikers. Of nieuwe gebruikers aanmaken.

Vandaar de keuze om enkele users en de Admin role toe te voegen door middel van seeding.

4.3.1 ADMIN USER EN ADMIN ROLE SEEDEN

Tijd om een admin user en admin role te seeden in onze database.

Voorzie volgende code in de `ApplicationDbContext` klasse:

```
public class ApplicationDbContext : IdentityDbContext<ApplicationUser>
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }

    protected override void OnModelCreating(ModelBuilder builder)
    {
        base.OnModelCreating(builder);

        const string AdminRoleId = "00000000-0000-0000-0000-000000000001";
        const string AdminRoleName = "Admin";

        const string AdminUserId = "00000000-0000-0000-0000-000000000001";
        const string AdminUserName = "admin@programmingintegration.be";
        const string AdminFirstname = "Mike";
        const string AdminLastname = "Admin";
        const string AdminUserPassword = "Test123?"; // For demo purposes only! Don't do this in real application!

        IPasswordHasher<ApplicationUser> passwordHasher = new PasswordHasher<ApplicationUser>(); // Identity password
        hasher

        ApplicationUser adminApplicationUser = new ApplicationUser
        {
            Id = AdminUserId,
            UserName = AdminUserName,
            NormalizedUserName = AdminUserName.ToUpper(),
            Email = AdminUserName,
            NormalizedEmail = AdminUserName.ToUpper(),
            EmailConfirmed = true,
            Firstname = AdminFirstname,
            Lastname = AdminLastname,
            SecurityStamp = "VVPCRDAS3MJWQD5CSW2GWPRADBXEZINA", //Random string
            ConcurrencyStamp = "c8554266-b401-4519-9aeb-a9283053fc58", //Random guid string
        };

        adminApplicationUser.PasswordHash = passwordHasher.HashPassword(adminApplicationUser, AdminUserPassword);

        builder.Entity<IdentityRole>().HasData(new IdentityRole
        {
            Id = AdminRoleId,
            Name = AdminRoleName,
            NormalizedName = AdminRoleName.ToUpper()
        });

        builder.Entity<ApplicationUser>().HasData(adminApplicationUser);

        builder.Entity<IdentityUserRole<string>>().HasData(new IdentityUserRole<string>
        {
            RoleId = AdminRoleId,
            UserId = AdminUserId
        });
    }
}
```

Voor het seeden dienen we de primary keys voor onze admin user en admin role eerst in een variabele te stoppen. De naam van de admin rol en admin user worden eerst in een constante variabele gestopt om dan in de object initialisatie de `.ToUpper()` methode te kunnen toevoegen bij de Normalized properties.

Het password dient ook gehashed te zijn in de database, hiervoor gebruiken we de implementatie van de `Identity PasswordHaser<TUser>` klasse.

4.3.2 MIGRATION EN DATABASE UPDATE

Voorzie nu een nieuwe database migration en doe hierna een update van je database.

We zien dat onze admin user, admin rol en de tussentabel onze seeding data bevat.

4.3.2.1 ASPNETUSERS

	Id	FirstName	LastName	DateOfBirth	UserName	NormalizedUserName	Email	NormalizedEmail	EmailConfirmed	PasswordHash
1	00000000-0000-0000-0000-000000000001	Mike	Admin	0001-01-01 00:00:00.0000000	admin@rateamovie.be	ADMIN@RATEAMOVIE.BE	admin@rateamovie.be	ADMIN@RATEAMOVIE.BE	1	AQAAAAEAACcQAAAAEPvTZSTocBfomWDDjCkDP69rh8S

4.3.2.2 ASPNETROLES

	Id	Name	NormalizedName	ConcurrencyStamp
1	00000000-0000-0000-0000-000000000001	Admin	ADMIN	232da504-d01b-4594-9b95-1945fcc48eea

4.3.2.3 ASPNETUSERROLES

	UserId	RoleId
1	00000000-0000-0000-0000-000000000001	00000000-0000-0000-0000-000000000001

4.4 USERSCONTROLLER ENKEL VOOR ADMIN

We zullen nu een `UsersController` toevoegen die enkel en alleen toegankelijk mag zijn voor users met een admin role.

Voeg een nieuwe `UsersController` toe in de `Controllers` folder met volgende code:

```
namespace Pri.Identity.Web.Controllers
{
    public class UsersController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }
    }
}
```

Aangezien deze controller enkel toegankelijk mag zijn voor users met de admin role plaatsen we het `Authorize` attribuut maar dan met de invulling van de rollen die toegang mogen krijgen. In ons geval dus enkel users met de admin rol.

```
[Authorize(Roles = "Admin")]
public class UsersController : Controller
{
    public IActionResult Index()
    {
        return View();
    }
}
```

Maak een nieuwe View aan en voorzie volgende:

```
<h1>Users</h1>

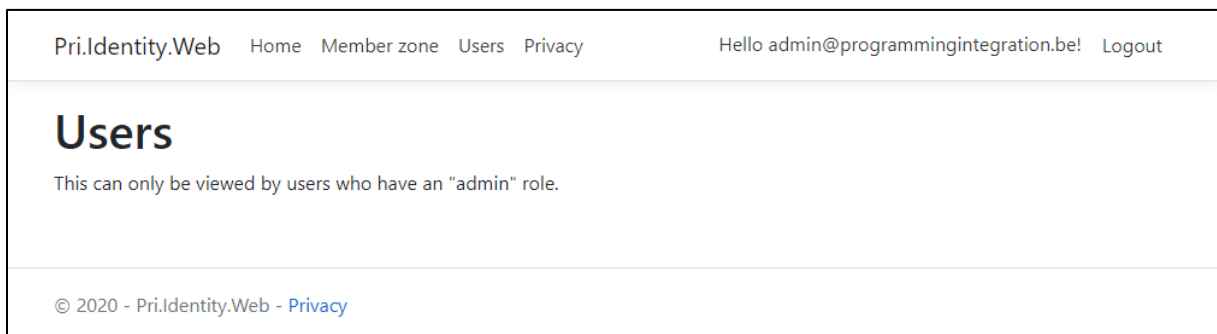
<p>This can only be viewed by users who have an "admin" role.</p>
```

Voorzie een nieuw menu item in de `_layout.cshtml` view:

```
<ul class="navbar-nav flex-grow-1">
  <li class="nav-item">
    <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Index">Home</a>
  </li>
  <li class="nav-item">
    <a class="nav-link text-dark" asp-area="" asp-controller="MembersZone" asp-action="Index">Member zone</a>
  </li>
  @if (Context.User.Identity.IsAuthenticated)
  {
    <li class="nav-item">
      <a class="nav-link text-dark" asp-area="" asp-controller="Users" asp-action="Index">Users</a>
    </li>
  }
  <li class="nav-item">
    <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Privacy">Privacy</a>
  </li>
</ul>
```

We testen dit even uit. Log even in met de user die een admin role heeft en surf naar <https://localhost:5001/Users>

We zien dat we toegang verkrijgen tot de view:



Log nu even uit en log je in met je user die geen admin role heeft en surf opnieuw naar de Users page. We merken dat we een NotFound error krijgen. Dit komt doordat we nog geen Access Denied view hebben aangemaakt.

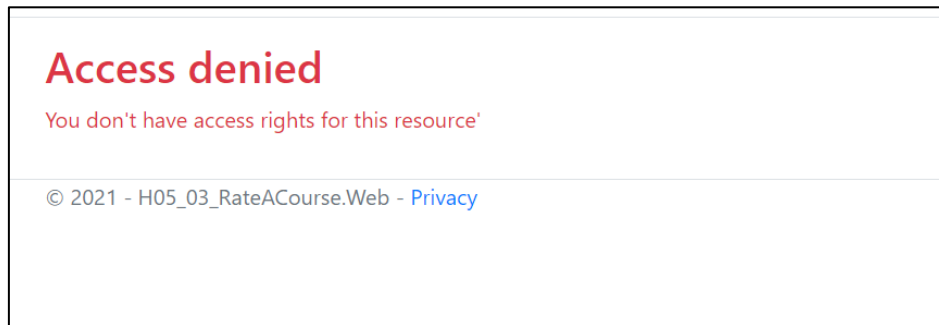
Voeg onderstaande methode toe aan de AccountController:

```
[HttpGet]
public IActionResult AccesDenied()
{
    return View();
}
```

Maak vervolgens een View aan voor deze methode en voeg onderstaande code eraan toe:

```
<h2 class="text-danger">Access denied</h2>
<p class="text-danger">You don't have access rights for this resource'</p>
```


Dit zal resulteren in onderstaande view:



Om ervoor te zorgen dat de gebruiker correct doorverwezen wordt naar de AccessDenied pagina dienen we nog wat code toe te voegen aan de `Program.cs`

```
//set path for AccessDenied
builder.Services.ConfigureApplicationCookie(options =>
{
    options.AccessDeniedPath = "/account/account/AccessDenied";
});
builder.Services.AddControllersWithViews();
```

Test nu de applicatie uit, je zal merken dat we geen NotFound pagina meer te zien krijgen maar wel onze Access Denied pagina.

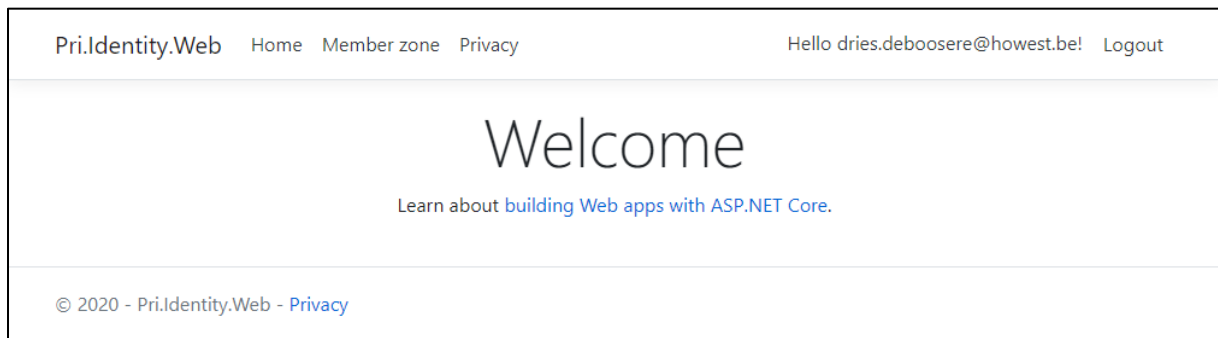
Om de gebruiker een betere user experience te geven zou het beter zijn om de link “Users” niet te tonen in het menu als deze er geen toegang tot heeft.

Daarom voegen we volgende if selectie toe:

```
<ul class="navbar-nav flex-grow-1">
...
    @if (User.IsInRole("Admin"))
    {
        <li class="nav-item">
            <a class="nav-link text-dark" asp-area="" asp-controller="Users" asp-action="Index">Users</a>
        </li>
    }
...
</ul>
```

In onze View kunnen we dus rechtstreeks de `User` aanspreken en hebben we toegang tot een `IsInRole()` methode die een boolean retourneert aan de hand van de role name die meegegeven wordt als parameter.

Als we dus ingelogd zijn als een user die geen admin is, zien we ook de link “Users” niet meer in het menu:



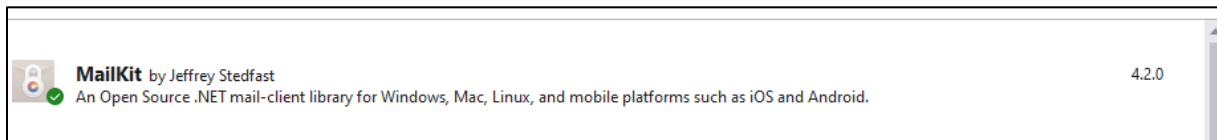
5 APPENDIX

De volgende twee onderdelen illustreren hoe je identity kan uitbreiden om een emailconfirmation te verzenden bij registratie en een eenvoudig gebruikersbeheer kan opzetten. Deze onderdelen zijn ter illustratie en horen niet bij de te kennen leerstof voor dit onderdeel.

5.1 EMAILCONFIRMATION

Nu we de basis van onze authenticatie geïmplementeerd hebben kunnen we de email confirmation aanpakken. Het is de bedoeling dat een nieuwe gebruiker zijn emailadres bevestigt om aan te tonen dat het effectief om een bestaand emailadres gaat. We doen dit door een bevestigingsmail te versturen naar het emailadres dat werd ingegeven bij de registratie met daarin een link naar een validatiepagina. Die link bevat het user id van de nieuwe gebruiker samen met een unieke token. Beiden zullen door het systeem gebruikt worden om de authenticiteit van het emailadres te verifiëren.

Om de emailverificatie te simuleren gaan we gebruik maken van MailKit, een NuGet package waarmee we vanuit een C# applicatie een email kunnen versturen. De eerste stap is dus het installeren van dit package:



Eens dit package geïnstalleerd is kunnen we van start met de implementatie.

5.1.1 EEN TOKEN GENEREREN

Om een emailverificatie uit te voeren hebben we een uniek token nodig die samen met de UserId van de nieuwe gebruiker gevalideerd zal worden. De `UserManager` service bevat hier de nodige methode:

```
_userManager.GenerateEmailConfirmationTokenAsync(user);
```

Deze methode zal op basis van de net aangemaakte user een uniek token genereren. Voeg onderstaande code toe aan de HttpPost Register methode van de AccountController:

```
var user = new ApplicationUser
{
    UserName = accountRegisterViewModel.Email,
    Firstname = accountRegisterViewModel.Firstname,
    Lastname = accountRegisterViewModel.Lastname,
    Email = accountRegisterViewModel.Email,
    EmailConfirmed = false
};
await _userManager.CreateAsync(user, accountRegisterViewModel.Password);
var token = await _userManager.GenerateEmailConfirmationTokenAsync(user);
```

Deze token zal later in het emailbericht geplaatst worden naar de nieuwe gebruiker.

5.1.2 DE VALIDATEEMAIL METHODE

Wanneer de gebruiker op de link zal klikken in de bevestigingsmail zal hij worden doorverwezen naar een methode die de emailvalidatie zal uitvoeren. Deze methode zal twee parameters ontvangen: een userId en het eerder gegenereerde token. Deze methode zal gebruik maken van de

`_userManager.VerifyUserTokenAsync` methode om de token te valideren. Plaats onderstaande code in de `AccountController`:

```
[HttpGet]
public async Task<IActionResult> ValidateEmail(string userId, string token)
{
    var user = await _userManager.FindByIdAsync(userId);
    if(user == null)
    {
        return NotFound();
    }
    if(await
_userManager.VerifyUserTokenAsync(user, TokenOptions.DefaultProvider, "EmailConfirmation", token))
    {
        await _userManager.ConfirmEmailAsync(user, token);
    }
    return View();
}
```

De `_userManager.VerifyUserTokenAsync` neemt vier parameters mee:

- De user: die we eerst gaan opvragen om te controleren of het om een bestaande user gaat.
- `TokenOptions.DefaultProvider`: Om aan te geven dat we de default provider gebruiken om de token te genereren.
- "EmailConfirmation": om aan te geven dat het een token om een emailconfirmation betreft.
- Token: het gegenereerde token dat gevalideerd dient te worden.

Deze methode retourneert een bool om aan te geven dat de validatie gelukt is. Indien dit true retourneert kunnen we door middel van de `await _userManager.ConfirmEmailAsync(user, token);` aanroep de useremail valideren in de database. Ook hier geven we de user en de token mee als parameters. Dit zal ervoor zorgen dat de `EmailConfirmed` kolom in de `AspNetUsers` table op True geplaatst wordt, waardoor de gebruiker zal kunnen inloggen op het systeem.

	Id	Firstname	Lastname	DateOfBirth	UserName	Normalize...	Email	Normalize...	EmailConfirmed	PasswordH...	!
▶	7b15855c80e2	Mileto	Di Marco	0001-01-01 ...	mileto.di.m...	MILETO.DI...	mileto.di.m...	MILETO.DI...	True	AQAAAAEA...	K

Tot slot maken we best een view voor onze `ValidateEmail` methode om een bericht te tonen aan onze gebruiker. Maak een view aan met onderstaande code:

```
<div class="row">
    <div class="col col-lg-8 mx-auto">
        <h5>Email confirmed</h5>
        <div class="alert alert-primary" role="alert">
            Your email has been confirmed. You can
            <a asp-controller="Account"
              asp-action="Login"
              asp-area="Account"
              class="alert-link">
                log in</a> if you like.
            </div>
        </div>
    </div>
</div>
```

Dit zal onderstaande pagina genereren:

Email confirmed

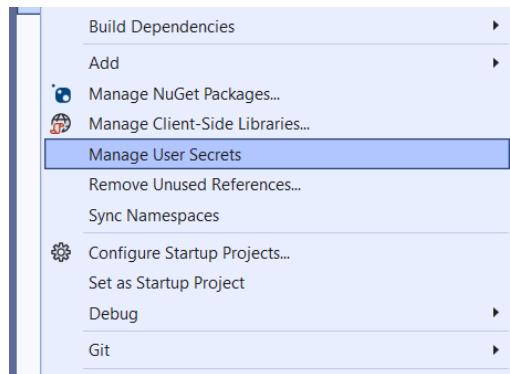
Your email has been confirmed. You can **log in** if you like.

5.1.3 EMAIL VERSTUREN MET MIMEKIT

Nu hoeven we enkel nog het emailbericht op te bouwen en te versturen met behulp van MimeKit. In een testomgeving kan je gerust gebruik maken van jouw eigen gmail, of ander, emailadres om te fungeren als smtp server. Als je een gmail account wenst te gebruiken zal je eerst een app password moeten aanmaken. Hoe je dit precies doet vind je op deze [link](#). Eens je het app paswoord hebt kan je het gaan gebruiken om de bevestigingsemail te laten versturen.

5.1.3.1 SECRETS.JSON

Een .Net Web app kan gevoelige informatie opslaan in een apart .json bestand zodat je deze gegevens niet in je code dient te plaatsen. Dit .json bestand heeft als naam secrets.json. Om de secrets.json file te openen rechtsklik je op je Web project en selecteer je "Manage User Secrets".



Het secrets.json bestand zal geopend worden. Plaats onderstaande code in het bestand en sla het op. Zorg ervoor dat je de Account en ApiKey waarden vervangt door je eigen gmail account en app password.

```
{
  "Email": {
    "Account": "<myAccount>@gmail.com",
    "ApiKey": "<myAppPassword>"
  }
}
```

De gegevens binnen een secrets.json file kunnen opgevraagd worden via de configuration manager. We injecteren hiervoor de `IConfiguration` interface in onze AccountController. Plaats de code in het geel bij jouw declaraties bovenaan de AccountController:

```
private readonly UserManager<ApplicationUser> _userManager;
private readonly SignInManager<ApplicationUser> _signInManager;
private readonly IConfiguration _configuration;

public AccountController(UserManager<ApplicationUser> userManager,
    SignInManager<ApplicationUser> signInManager,
    IConfiguration configuration)
{
    _userManager = userManager;
    _signInManager = signInManager;
    _configuration = configuration;
}
```

5.1.3.2 DE CONFIRMATIONLINK

Vooraleer we de email kunnen versturen maken we eerst de confirmation link aan. Die bestaat uit drie delen:

- Het schema: http of https
- De host: in ons geval is dat de localhost:<poort>
- De controller en action method: Account en ValidateEmail
- De nodige parameters: userId, token en ook de Area naam

Om deze link op een correcte manier op te bouwen maken we gebruik van de ingebouwde `LinkGenerator` en `IHttpContextAccessor`.

We maken eerst de `IHttpContextAccessor` beschikbaar voor dependency injection in de `Program.cs`.

```
builder.Services.AddHttpContextAccessor();
```

Vervolgens kunnen we deze injecteren in onze `AccountController`. We injecteren dan ook meteen de `LinkGenerator` class.

```
private readonly UserManager<ApplicationUser> _userManager;
private readonly SignInManager<ApplicationUser> _signInManager;
private readonly IHttpContextAccessor _httpContextAccessor;
private readonly LinkGenerator _linkGenerator;
private readonly IConfiguration _configuration;

public AccountController(UserManager<ApplicationUser> userManager,
    SignInManager<ApplicationUser> signInManager,
    IHttpContextAccessor httpContextAccessor, LinkGenerator linkGenerator,
    IConfiguration configuration)
{
    _userManager = userManager;
    _signInManager = signInManager;
    _httpContextAccessor = httpContextAccessor;
    _linkGenerator = linkGenerator;
    _configuration = configuration;
}
```

Nu zijn beide classes toegankelijk voor onze controller. Voeg onderstaande code toe aan de `HttpPost Register` methode, net onder de code om de token aan te maken:

```
var token = await _userManager.GenerateEmailConfirmationTokenAsync(user);
var confirmationLink = _linkGenerator.GetUriByAction
(
    action: "ValidateEmail",
    controller: "Account",
    scheme: _httpContextAccessor.HttpContext.Request.Scheme,
    host: _httpContextAccessor.HttpContext.Request.Host,
    values: new { Area = "Account", userId = user.Id, token = token }
);
```

De `GetUriByAction` methode genereert een correcte link op basis van de meegeleverde parameters. De action en controller parameters spreken voor zich.

De scheme(https) en host (localhost:<poort>) worden opgehaald door middel van de `HttpContextAccessor`. De values parameter bevat de nodige userId en token, en ook de Areanaam(Account). Op deze manier wordt onderstaande link opgebouwd:

<https://localhost:44347/Account/Account/ValidateEmail?userId=0c172993-5175-4cdc-82f7-70c72f1415c8&token=CfDJ8Pe5Vw7%2B6R5EqYYOhPsrOJ43QHq5RxLFKsuz7PrUNy4KLAiDR5caZlzpCoy%2FBAJslZshDp9trOURDQWJrEpJJX60SLbwXSE9JQ6Aw%2FsW%2FyELG6k0JB5MN4VF4VM9L7C8w0ZgR2t8YrptwnkNZKqQl7%2F0i9Bsmc6c%2Bx%2Fd6HHeiEesXVSmPG6V6k7bRjVOIBchcm%2FdAigh2%2F%2B6REEk4O2qe6JPjWUZYawi20d5F1oRISBJca6Fenh3Cuo7PzP8CDHw%3D%3D>

5.1.3.3 E-MAIL VERSTUREN

Nu we de token en de confirmatielink hebben opgebouwd kunnen we verder met het laatste deel: het effectief versturen met behulp van MimeKit. Plaats onderstaande code inde `HttpPost` register methode, net onder de confirmation link code. Let erop dat we voor de accountgegevens onze `_configuration` property zullen aanspreken:

```
var email = new MimeMessage();
email.From.Add(MailboxAddress.Parse(_configuration["Email:Account"]));
email.To.Add(MailboxAddress.Parse(user.Email));
email.Subject = "Confirm your emailaddress";
email.Body = new TextPart(TextFormat.Html)
{
    Text = $"<h5>Please confirm your emailaddress</h5>" +
        $"<p>Please confirm your emailaddress by clicking " +
        $"<a href='{confirmationLink}'>here</a>";
};
```

We maken een instantie aan van de `MimeMessage` klasse om de email op te bouwen. Deze bevat onderstaande zaken:

- De `From.Add` methode: om de verzender email(s) toe te voegen. In dit voorbeeld gebruik ik de waarde van de `Email:Account` entry in de `secrets.json`.
- De `To.Add` methode: om de ontvanger(s) toe te voegen. In dit geval zal dit het emailadres van de nieuwe gebruiker zijn.
- De `Subject` property: het onderwerp.
- De `Body` property: bevat de inhoud van onze mail met de vraag om op de confirmation link te klikken en de confirmation link als link. Merk op dat we hier een `TextPart` object dienen te initialiseren met als parameter `TextFormat.Html` om de email in html formaat te kunnen versturen.

Na het opstellen van het bericht gaan we een instantie van de `SmtpClient` klasse initialiseren. Hiermee kunnen we het emailbericht versturen. Voeg onderstaande code toe, net na de vorige code.

```
using var smtpClient = new SmtpClient();
await smtpClient.ConnectAsync("smtp.gmail.com", 465,
SecureSocketOptions.SslOnConnect);
await smtpClient.AuthenticateAsync(_configuration["Email:Account"],
_configuration["Email:ApiKey"]);
var result = await smtpClient.SendAsync(email);
await smtpClient.DisconnectAsync(true);
```

De methodes in het geel gemarkeerd handelen het versturen af in de correcte volgorde

1. SmtpClient initialiseren
2. Connecteren met de gmail smtp
3. Authenticeren met jouw gmailadres en het app paswoord.
4. Email versturen
5. Disconnecteren van de gmail smtp server.

Uiteraard dien je in dit voorbeeld je eigen gmail en app password toe te voegen.

De volledige HttpPost Register code vind je nog eens hieronder:

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Register(AccountRegisterViewModel accountRegisterViewModel)
{
    //errors from form validation
    if (!ModelState.IsValid)
    {
        return View(accountRegisterViewModel);
    }
    if (await _userManager.FindByNameAsync(accountRegisterViewModel.Email) == null)
    {
        var user = new ApplicationUser
        {
            UserName = accountRegisterViewModel.Email,
            Firstname = accountRegisterViewModel.Firstname,
            Lastname = accountRegisterViewModel.Lastname,
            Email = accountRegisterViewModel.Email,
            EmailConfirmed = false
        };
        await _userManager.CreateAsync(user, accountRegisterViewModel.Password);
        var token = await _userManager.GenerateEmailConfirmationTokenAsync(user);
        var confirmationLink = _linkGenerator.GetUriByAction
        (
            action: "ValidateEmail",
            controller: "Account",
            scheme: _httpContextAccessor.HttpContext.Request.Scheme,
            host: _httpContextAccessor.HttpContext.Request.Host,
            values: new { Area = "Account", userId = user.Id, token = token }
        );
        var email = new MimeMessage();
        email.From.Add(MailboxAddress.Parse("admin@rateACourse.com"));
        email.To.Add(MailboxAddress.Parse(user.Email));
        email.Subject = "Confirm your emailaddress";
        email.Body = new TextPart(TextFormat.Html)
        {
            Text = $"<h5>Please confirm your emailaddress</h5>" +
                $"<p>Please confirm your emailaddress by clicking " +
                $"<a href='{confirmationLink}'>here</a>";
        };
        using var smtpClient = new SmtpClient();
        await smtpClient.ConnectAsync("smtp.gmail.com", 587, SecureSocketOptions.StartTls);
        await smtpClient.AuthenticateAsync("...@gmail.com", "<your_app_password>");
        var result = await smtpClient.SendAsync(email);
        await smtpClient.DisconnectAsync(true);
        return RedirectToAction(nameof(Registered));
    }
    ModelState.AddModelError("Email", "This email is already taken!");
    return View(accountRegisterViewModel);
}
```

Deze code zou uiteraard best opgesplitst worden in afzonderlijke service klassen mocht je dit toepassen in een eigen project.

5.2 IDENTITY CRUD VOOR USERS EN ROLES

Identity biedt out-of-the-box enkele klassen, methodes en properties aan die gebruikt kunnen worden om gebruikersbeheer te implementeren.

Wens je dus data op te halen of te manipuleren omtrent je users, gebruik dan de klassen aangereikt door Identity. Wens je CRUD acties uit te voeren op je users; gebruik dan de `userManager<T>` klasse en NIET de `ApplicationDbContext` klasse.

We zullen er hier enkele overlopen. Hou er echter rekening mee dat er nog véél meer zijn dan diegene die we even overlopen.

5.2.1 IDENTITYUSER

Naam	Beschrijving
Id	Deze property bevat de ID van de user.
UserName	Deze property bevat de username van de user.
Email	Deze property bevat het email adres van de user.

Nog meer methodes en properties van de **IdentityUser** zijn te vinden in de Microsoft Docs.

5.2.2 USERMANAGER<TUSER>

Waar TUser staat voor je user class type.

Naam	Beschrijving
Users	Deze property geeft je een lijst van alle users in de database.
FindByIdAsync(id)	Deze methode bevraagt de database en bezorgt je het user object met de specifieke ID.
CreateAsync(user)	Deze methode slaat een nieuwe user op in de database <u>zonder password</u> .
CreateAsync(user, password)	Deze methode slaat een nieuwe user op in de database <u>met het gespecificeerde password</u> .
UpdateAsync(user)	Deze methode update een bestaande user aan in de database.
DeleteAsync(user)	Deze methode verwijdert een user uit de database.
AddToRoleAsync(TUser, String)	Voegt een user toe aan een role
RemoveFromRoleAsync(TUser, String)	Verwijdert een user uit een role

Zoals je ziet heb je hier de nodige methodes om CRUD acties uit te voeren op je Users.

Nog meer methodes en properties van de **userManager** zijn te vinden in de Microsoft Docs.

5.2.3 ROLEMANAGER<TRole>

Waar TRole staat voor je IdentityRole type.

Naam	Beschrijving
Roles	Deze property geeft je een IQueryable collectie van alle roles.
FindByIdAsync(id)	Deze methode bevraagt de database en bezorgt je het role object met de specifieke ID.
FindByNameAsync(name)	Deze methode bevraagt de database en bezorgt je het role object met de specifieke role naam.
CreateAsync(TRole)	Maakt een role aan.
UpdateAsync(TRole)	Past een role aan.
DeleteAsync(TRole)	Deze methode verwijdert een role uit de database.
GetRoleIdAsync(TRole)	Krijg de role id van de IdentityRole
GetRoleNameAsync(TRole)	Krijg de role naam van de IdentityRole
RoleExistsAsync(roleName)	Deze methode stuurt je een boolean terug of een rolennaam al of niet bestaat.

5.2.4 IDENTITYRESULT

Het resultaat van de `userManager.CreateAsync(user, password)` methode is een `Task<IdentityResult>` object. Deze geeft de uitkomst van de create operatie, gebruik makend van volgende properties:

Naam	Beschrijving
Succeeded	Returns true als de operatie succesvol was.
Errors	Retourneert een collective van IdentityError objecten die de errors beschrijven die zijn gebeurd tijdens het uitvoeren van de operatie. Elk IdentityError object bevat een Description property dat het probleem omschrijft.

Voorbeeld:

```
if (result.Succeeded)
{
    return RedirectToAction("List");
}
foreach (IdentityError err in result.Errors)
{
    ModelState.AddModelError("", err.Description);
}
```

Nog meer methodes en properties van de **IdentityResult** zijn te vinden in de Microsoft Docs.

5.2.5 SIGNINMANAGER<TUSER>

Waar TUser staat voor je user class type.

Naam	Beschrijving
SignInAsync(user, isPersistent)	Deze methode logt de user in.
SignOutAsync()	Deze methode logt de gebruiker uit.
PasswordSignInAsync(userName, password, isPersistent, lockoutOnFailure)	Deze methode logt de user in met username en password.

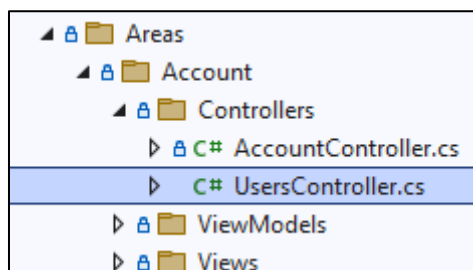
Nog meer methodes en properties van de **SignInManager** zijn te vinden in de Microsoft Docs.

5.2.6 EENVOUDIG ROLLENBEHEER

Om duidelijk aan te tonen hoe de Identity serviceklassen gebruikt kunnen worden om rollen te beheren schrijven we een kort voorbeeld uit. We hernemen de voorbeeldapplicatie Rate A Course en we bouwen een eenvoudig rollenbeheersysteem uit.

5.2.6.1 USERSCONTROLLER

Als eerste stap verwijderen we de UsersController die we in het vorige voorbeeld aangemaakt hebben. Vervolgens maken we een nieuwe UsersController in de Account area aan.



We declareren en injecteren de nodige Serviceklassen om het Identity Platform aan te spreken:

```
public class UsersController : Controller
{
    private readonly UserManager<ApplicationUser> _userManager;
    private readonly RoleManager<IdentityRole> _roleManager;

    public UsersController(UserManager<ApplicationUser> userManager,
        RoleManager<IdentityRole> roleManager)
    {
        _userManager = userManager;
        _roleManager = roleManager;
    }
    ...
}
```

De **UserManager** zal gebruikt worden om de users op te halen en om rollen toe te wijzen of te verwijderen. De **RoleManager** gebruiken we om rollen op te halen en toe te voegen. Merk op dat de **RoleManager** een

`IdentityRole` als generic type toegewezen krijgt, dit is de default klasse binnen Identity om een rol te implementeren.

5.2.6.2 DE INDEX METHODE

De Index methode zal in ons voorbeeld een lijst van users displayen met hun bijhorende rollen. We maken hiervoor eerst een `UserViewModel` aan die model zal staan voor een gebruiker. Maak deze klasse aan in de ViewModels map van de Account area en plaats er onderstaande code in:

```
public class UserViewModel : BaseViewModel
{
    public new string Id { get; set; }
    public IList<string> Roles { get; set; }
}
```

Deze klasse zal uiteindelijk gebruikt worden door het viewmodel van onze Index methode. De Name property wordt overgeërfd van de `BaseViewModel` klasse en we declareren een aangepaste Id property door middel van het `new` keyword. Dit om de string Id van onze `ApplicationUser` te kunnen toewijzen (het `BaseViewModel` bevat immers een long Id).

Vervolgens maken we een `UsersIndexViewModel` aan, deze klasse zal een lijst bevatten van het vorige `UserViewModel`.

```
public class UsersIndexViewModel
{
    public IEnumerable<UserViewModel> Users { get; set; }
}
```

We hebben uiteraard ook een view nodig om het overzicht van de gebruikers te displayen. Maak een View aan en plaats er onderstaande code in.

```
@model UsersIndexViewModel
<div class="row">
    <div class="col col-lg-10 mx-auto">
        <a asp-area="Account"
            asp-controller="Users"
            asp-action="CreateRole"
            class="btn btn-outline-info mb-2">New role</a>
        <table class="table table-hover table-responsive-lg">
            <thead>
                <tr>
                    <th>Name</th>
                    <th>Roles</th>
                    <th>Add to role</th>
                </tr>
            </thead>
            <tbody>
                @foreach (var user in Model.Users)
                {
                    <tr>
                        <td>@user.Name</td>
                        <td>
                            @foreach (var role in user.Roles)
                            {
                                <div> @role <a asp-area="Account"
                                    asp-controller="Users"
                                    asp-action="ConfirmRemoveUserFromRole"
                                    asp-route-id="@user.Id"
                                    asp-route-role="role"
                                    class="alert-link">Remove</a></div>
                            }
                        </td>
                    </tr>
                }
            </tbody>
        </table>
    </div>
</div>
```

```

        </td>
        <td>
            <a asp-area="Account"
               asp-controller="Users"
               asp-action="AddRole"
               asp-route-id="@user.Id"
               class="btn btn-outline-info">Add to role</a>
        </td>
    </tr>
}
</tbody>
</table>
</div>
</div>

```

Dit zal resulteren in het volgende scherm:

New role		
Name	Roles	Add to role
Mike Admin	Admin Remove	Add to role

Merk op dat er een link is om een gebruiker uit een rol te verwijderen en een knop om een rol toe te voegen aan een gebruiker. Er is ook een knop voorzien om een nieuwe rol toe te voegen aan het systeem. Die functionaliteiten worden later geïmplementeerd.

Nu gaan we verder met de implementatie van de Index methode. Plaats de volgende code in de Index methode van de UsersController:

```

public IActionResult Index()
{
    UsersIndexViewModel usersIndexViewModel = new UsersIndexViewModel();
    var users = _userManager.Users.ToList();
    usersIndexViewModel.Users = users.Select
    (
        u => new UserViewModel
        {
            Id = u.Id,
            Name = $"{u.Firstname} {u.Lastname}",
            Roles = _userManager.GetRolesAsync(u).Result
        }
    );
    return View(usersIndexViewModel);
}

```

De code in het geel zorgt voor het ophalen van de users en het ophalen van de bijhorende rollen.

Als laatste wijzig je de code die de link naar `UserController` displayt in de `_Layout` file naar:

```

@if (Context.User.IsInRole("Admin"))
{
    <li class="nav-item">
        <a class="nav-link text-dark" asp-area="Account" asp-controller="Users" asp-
        action="Index">Users</a>
    </li>
}

```

```
}
```

Zodat de knop in de navigatiebalk naar de correcte area en controller verwijst.

Start nu de applicatie op, log in met de admingebruiker en klik op de users link. Je zou een overzicht van de gebruikers moeten zien. Indien je nog geen nieuwe gebruikers hebt geregistreerd zal je hier enkel de Admin gebruiker in de lijst zien staan.

5.2.7 NIEUWE ROL TOEVOEGEN

De knop om een nieuwe rol toe te voegen verwijst naar een methode met als naam `CreateRole`. We maken eerst een passend viewmodel aan met als naam `UsersCreateRoleViewModel`.

```
public class UsersCreateRoleViewModel
{
    [Required]
    public string Role { get; set; }
    public IEnumerable<string> Roles { get; set; }
}
```

Dit viewmodel bevat een lijst van reeds bestaande Roles en een property om een nieuwe Role toe te voegen. Vervolgens maken we de `CreateRole` action methode aan:

```
[HttpGet]
public async Task<IActionResult> CreateRole()
{
    var roles = await _roleManager.Roles.ToListAsync();
    UsersCreateRoleViewModel usersCreateRoleViewModel = new();
    usersCreateRoleViewModel.Roles = roles.Select(r => r.Name);
    return View(usersCreateRoleViewModel);
}
```

We halen eerste een lijst van bestaande roles op en geven die door aan de view.

Maak nu de bijhorende view aan en plaats er onderstaande code in:

```
@model UsersCreateRoleViewModel
<div class="row">
    <div class="col col-lg-5 mx-auto">
        <h4>Add role</h4>
        <form Area="Account" asp-controller="Users" asp-action="CreateRole"
method="post">
            <div class="form-group">
                <label asp-for="Role"></label>
                <input asp-for="Role" class="form-control">
                <span asp-validation-for="Role" class="text-danger"></span>
            </div>
            <button type="submit" class="btn btn-primary">Add</button>
        </form>
    </div>
</div>
<div class="row">
    <div class="col col-lg-5 mx-auto">
        <h4>Roles</h4>
        <table class="table table-hover">
            <thead>
                <tr>
                    <th>Role</th>
                </tr>
            </thead>
            <tbody>
                @foreach (var role in Model.Roles)
                {
```

```

        <tr>
            <td>@role</td>
        </tr>
    }
</tbody>
</table>
</div>
</div>

```

Dit zal resulteren in onderstaand eenvoudig scherm:

The screenshot shows a web form titled "Add role". It has a text input field for "Role" and a blue "Add" button. Below the button is a section titled "Roles" which contains a table with a single row labeled "Admin".

Maak nu de [HttpPost] methode CreateRole aan en plaats er onderstaande code in:

```

[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> CreateRole(UsersCreateRoleViewModel
usersCreateRoleViewModel)
{
    if(!ModelState.IsValid)
    {
        return View(usersCreateRoleViewModel);
    }
    if(await _roleManager.RoleExistsAsync(usersCreateRoleViewModel.Role))
    {
        ModelState.AddModelError("Role", "Role exists");
        usersCreateRoleViewModel.Roles = await _roleManager.Roles.Select(r =>
r.Name).ToListAsync();
        return View(usersCreateRoleViewModel);
    }
    await _roleManager.CreateAsync(new IdentityRole
    {
        Name = usersCreateRoleViewModel.Role,
    });
    return RedirectToAction("CreateRole");
}

```

Let op de code in geel: eerste controleren we of de role reeds bestaat, en we genereren een ModelState error indien dit het geval is. Wanneer de role niet bestaat wordt die toegevoegd gebruik makende van de `RoleManager.CreateAsync()` methode, deze ontvangt uiteraard een `IdentityRole` object. Tot slot Redirecten we naar de CreateRole [HttpGet] aciotn methode en de nieuwe rol zal zichtbaar zijn in de lijst.

Start de applicatie uit, log in als admin en test de nieuwe functionaliteit uit. Voeg een nieuwe role toe met als naam User. Als alles goed gaat verschijnt die nieuwe role in de lijst. Test ook de validatieregels uit.



5.2.8 EEN ROLE TOEVOEGEN AAN EEN GEBRUIKER

Nu kunnen we verder met de implementatie van de AddRole action method die een rol aan een gebruiker zal toewijzen. Maak eerst een viewmodel aan met als naam `UsersAddRoleViewModel`:

```
public class UsersAddRoleViewModel
{
    [HiddenInput]
    public string Id { get; set; }
    public IEnumerable<SelectListItem> Roles { get; set; }
    [Display(Name = "Roles")]
    public string RoleId { get; set; }
}
```

De view zal een simpele drop down displayen met de beschikbare rollen. Vervolgens maak je de HttpGet action methode aan die de lijst zal opbouwen en doorgeven aan de view:

```
[HttpGet]
public async Task<IActionResult> AddRole(string id)
{
    var roles = await _roleManager.Roles.ToListAsync();
    UsersAddRoleViewModel usersAddRoleViewModel = new();
    usersAddRoleViewModel.Roles = roles.Select(r => new SelectListItem
    {
        Text = r.Name,
        Value = r.Name
    });
    usersAddRoleViewModel.Id = id;
    return View(usersAddRoleViewModel);
}
```

De roles worden opgehaald en in het viewmodel gestopt. Ook het Id van de user wordt meegegeven aan de view. Dit zal resulteren in onderstaand scherm:



Vervolgens maken we de HttpPost Addrole methode aan:

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> AddRole(UsersAddRoleViewModel usersAddRoleViewModel)
{
    var user = await _userManager.FindByIdAsync(usersAddRoleViewModel.Id);
    if(user == null
        ||
        !await _roleManager.RoleExistsAsync(usersAddRoleViewModel.RoleId))
    {
        return NotFound();
    }
    await _userManager.AddToRoleAsync(user, usersAddRoleViewModel.RoleId);
    return RedirectToAction("Index");
}
```

Eerst controleren we of de user en de role effectief bestaan. Vervolgens wordt de user toegewezen aan de role met behulp van de `Usermanager.AddTorRoleAsync()` methode waarnaar we een redirect uitvoeren naar de Index methode die de user lijst displayt.

Start de applicatie op en test uit. Voeg de User role toe aan de admin gebruiker.

New role		
Name	Roles	Add to role
Mike Admin	Admin Remove User Remove	Add to role

5.2.9 EEN ROLE VERWIJDEREN VAN EEN GEBRUIKER

Tot slot werken we de functionaliteit uit om een bepaalde role van een user af te nemen. We maken hiervoor eerst een `UsersRemoveUserFromRoleViewModel` klasse aan die her userId en de roleId zal bijhouden van de User en de Role die niet meer bij elkaar mogen horen.

```
public class UsersRemoveUserFromRoleViewModel
{
    [HiddenInput]
    public string UserId { get; set; }
    [HiddenInput]
    public string RoleId { get; set; }
}
```

Vervolgens maken we ene action methode met als naam ConfirmRemoveUserFromRole die een string userId en een string roleId als parameters heeft.

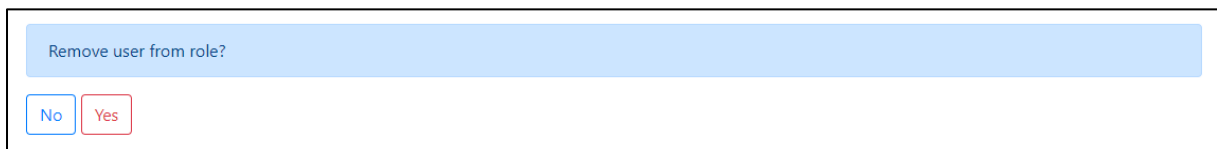
```
[HttpGet]
public IActionResult ConfirmRemoveUserFromRole(string userId, string roleId)
{
    UsersRemoveUserFromRoleViewModel usersRemoveUserFromRoleViewModel = new();

    return View(usersRemoveUserFromRoleViewModel);
}
```

Deze methode zal een confirmation message tonen aan de gebruiker met de vraag of de rol echt verwijderd moet worden van de gebruiker. De view hiervoor is zo opgebouwd:

```
@model UsersRemoveUserFromRoleViewModel
<form asp-area="Account" asp-controller="Users" asp-action="RemoveUserFromRole"
method="post">
    <div class="alert alert-primary" role="alert">
        Remove user from role?
    </div>
    <a asp-controller="Users" asp-action="Index" class="btn btn-outline-
primary">No</a>
    <input asp-for="UserId" />
    <input asp-for="RoleId" />
    <button type="submit" class="btn btn-outline-danger">Yes</button>
</form>
```

En ziet er zo uit in runtime:



The screenshot shows a light blue alert box with the text "Remove user from role?". Below the text are two buttons: "No" (outlined) and "Yes" (outlined in red).

De form submit action stuurt de gebruiker door, in het geval dat er op Yes wordt geklikt, naar de action methode `RemoveUserFromRole`. Deze methode werken we als onderstaand uit:

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> RemoveUserFromRole(UsersRemoveUserFromRoleViewModel
usersRemoveUserFromRoleViewModel)
{
    var user = await
_userManager.FindByIdAsync(usersRemoveUserFromRoleViewModel.UserId);
    var role = await
_roleManager.FindByIdAsync(usersRemoveUserFromRoleViewModel.RoleId);
    if(user == null || role == null)
    {
        return NotFound();
    }
    await _userManager.RemoveFromRoleAsync(user, role.Name);
    return View("Index");
}
```

Let erop dat we eerst de User en de Role ophalen om een null controle uit te voeren, en dat we vervolgens de User vanuit de Role verwijderen met een aanroep naar de `_userManager.RemoveFromRoleAsync(user, role.Name)` methode.

Start de applicatie op, log in met jouw admingebruiker en test uit of je de User role kan verwijderen van jouw admingebruiker.

Uiteraard zou het aangewezen zijn om de functionaliteiten voor het rollenbeheer in een daarvoor voorziene service klasse te plaatsen die door de controller aangesproken kan worden met gebruik van dependency injection.



Code repository

De volledige broncode van dit hoofdstuk is te vinden op

 `git clone` <https://classroom.github.com/a/r5rPCY8o>

